

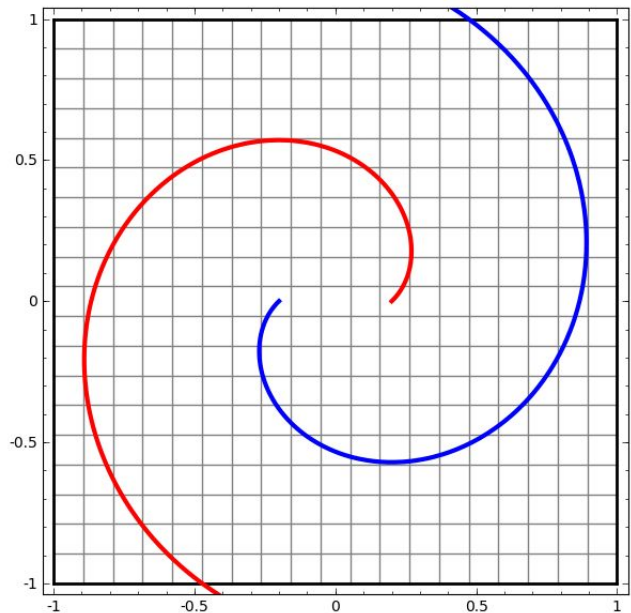
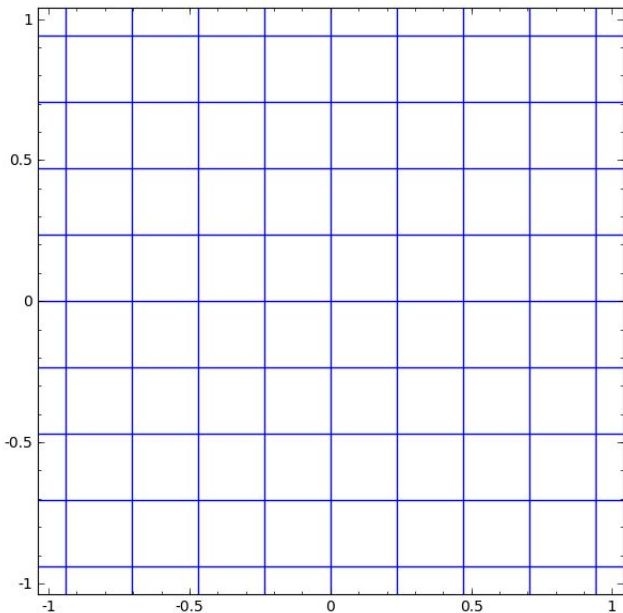
Tecnología Digital VI

Activación

Licenciatura en Tecnología Digital - UTDT

Capacidad de las NN

¿Qué son capaces de hacer las Redes Neuronales?



Funciones de activación

El efecto final de una red neuronal es la composición de los efectos de cada capa. Si no hay funciones de activación, cada capa realiza combinaciones lineales de los inputs con los pesos (una matriz que multiplica los inputs). Componer muchas transformaciones lineales (o multiplicar muchas veces por matrices) no deja de ser otra transformación lineal (o una gran matriz final).

Necesitamos las funciones de activación (y que sean no lineales!) para romper la linealidad total en la red.

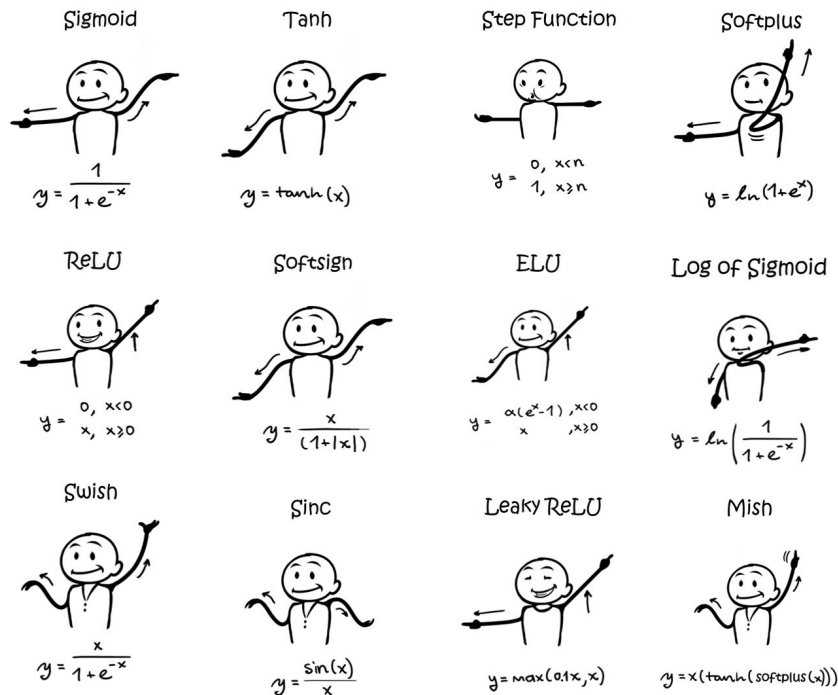
Funciones de activación

¿Por qué hablamos de “funciones de activación” en plural? ¿No puedo poner alguna función no lineal y listo?

- En la teoría: casi que sí, mientras que pongamos cualquier función no polinómica. “Multilayer feedforward networks with a nonpolynomial activation function can approximate any function”
- En la práctica: diferentes opciones de funciones no polinómicas tienen impacto en los tiempos de entrenamiento, convergencia, problemas adicionales, etc.

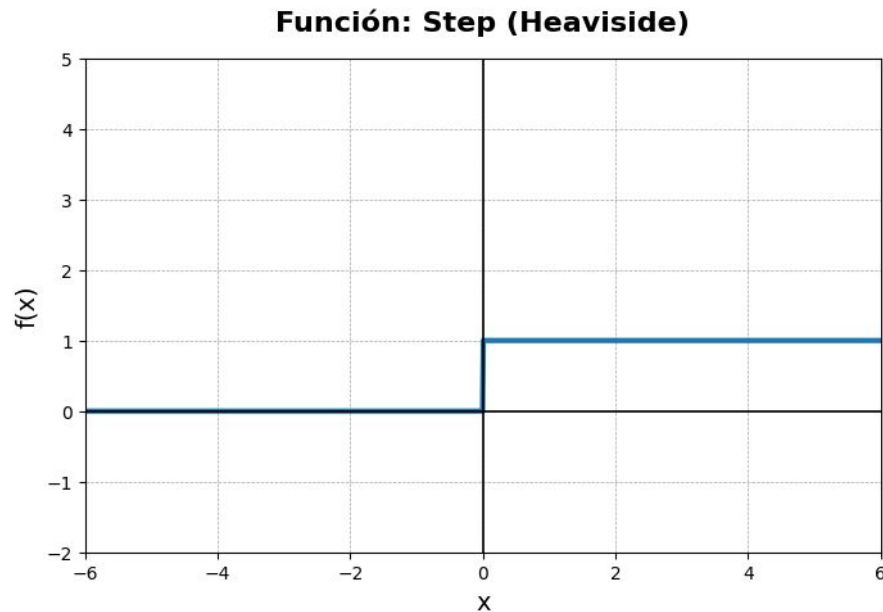
Funciones de activación

En la práctica existen varias funciones de activación posibles. Más allá de ciertas tendencias, en general la aparición de nuevas funciones intentan resolver inconvenientes prácticos presentes en las funciones anteriores.



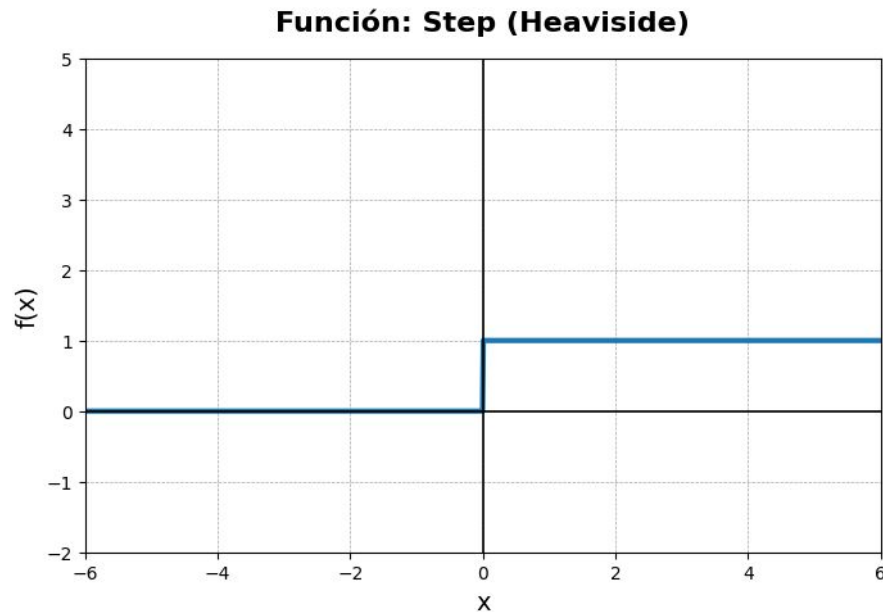
Heaviside Step Function

- Definición: $f(x) = 1$ si $x \geq 0$, de lo contrario 0
- Cuándo: Entre 1940 y 1950.
- Por qué: Modelaba el *spike* de una neurona biológica.



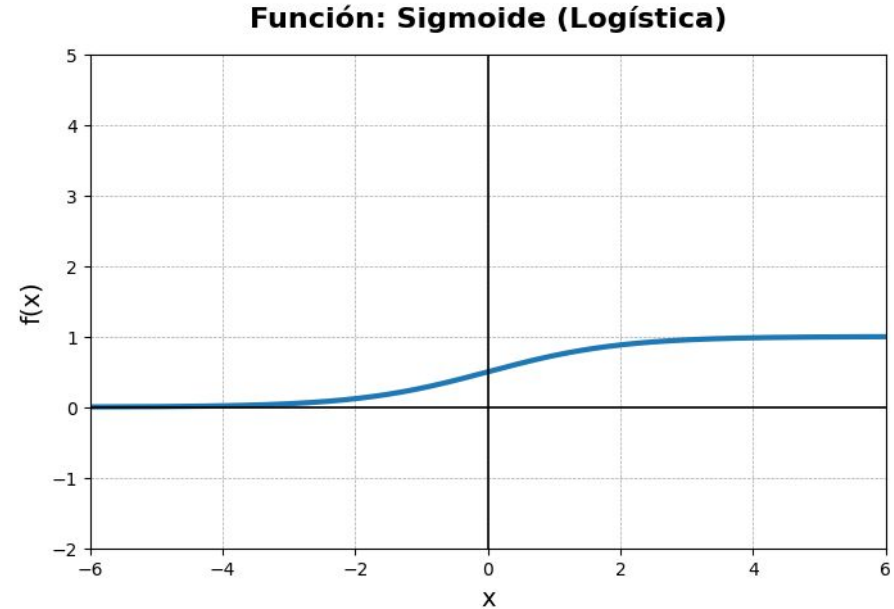
Heaviside Step Function

- Ventajas:
 - Simple de entender e implementar.
 - Simple de computar.
 - Útil para clasificación binaria (como activación en la capa de salida).
- Desventajas:
 - Derivada nula en todo el dominio, excepto en el 0 donde no es derivable.
 - No hay gradualidad, muy sensible.
- Alternativas: La imposibilidad de aplicar backpropagation.



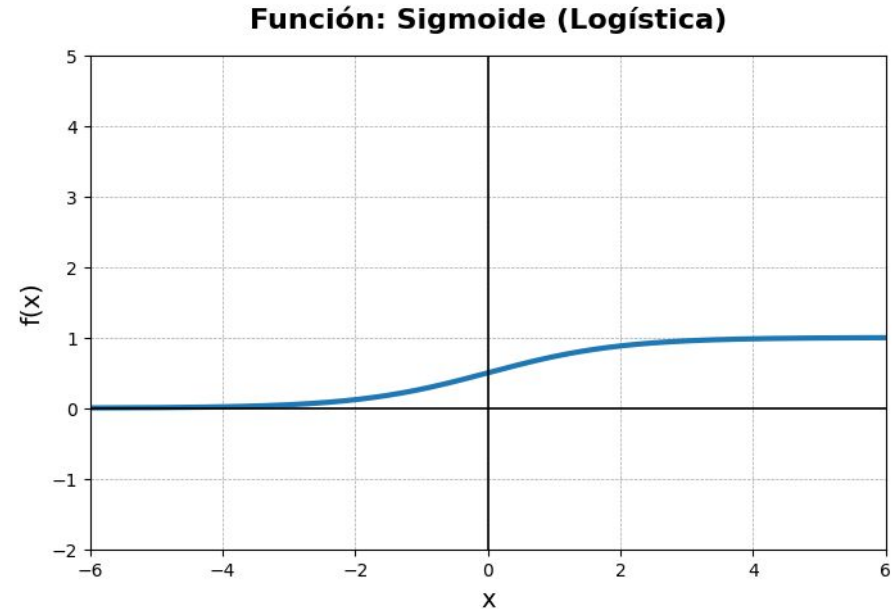
Sigmoidea o Logística

- Definición: $f(x) = \frac{1}{1+e^{-x}}$
- Cuándo: Por los 80, junto con Backpropagation.
- Por qué: para tener una función entre 0 y 1 suave y diferenciable.



Sigmoidea o Logística

- Ventajas:
 - Derivada en todo el dominio.
 - Derivada simple: $\text{sig} \cdot (1 - \text{sig})$.
 - La salida está entre 0 y 1 (sirve para probabilidades).
- Desventajas:
 - En los extremos sufre de *vanishing gradients*.
 - La salida no está centrada en cero (*zig-zag*).
 - Hay que calcular una exponencial.
- Alternativas: el mayor factor para buscar una alternativa fue el problema del *zig-zag*.



Zero-Centering o Zig-Zag

Las funciones que tienen siempre el mismo signo tienen un efecto indeseado en los algoritmos de optimización como Gradient Descent.

Por como terminan siendo las actualizaciones de los pesos, se produce un efecto de *zig-zag* en la optimización que ralentiza la convergencia.

4.3 Normalizing the Inputs

Convergence is usually faster if the average of each input variable over the training set is close to zero. To see this, consider the extreme case where all the inputs are positive. Weights to a particular node in the first weight layer are updated by an amount proportional to δx where δ is the (scalar) error at that node and x is the input vector (see equations (5) and (10)). When all of the components of an input vector are positive, all of the updates of weights that feed into a node will be the same sign (i.e. $\text{sign}(\delta)$). As a result, these weights can only all decrease or all increase *together* for a given input pattern. Thus, if a weight vector must change direction it can only do so by zigzagging which is inefficient and thus very slow.

In the above example, the inputs were all positive. However, in general, any shift of the average input away from zero will bias the updates in a particular direction and thus slow down learning. Therefore, it is good to shift the inputs so that the average over the training set is close to zero. This heuristic should be applied at all layers which means that we want the average of the *outputs* of a node to be close to zero because these outputs are the inputs to the next layer [19]. This problem can be addressed by coordinating how the inputs are transformed with the choice of sigmoidal activation function. Here we discuss the input transformation. The discussion of the sigmoid follows.

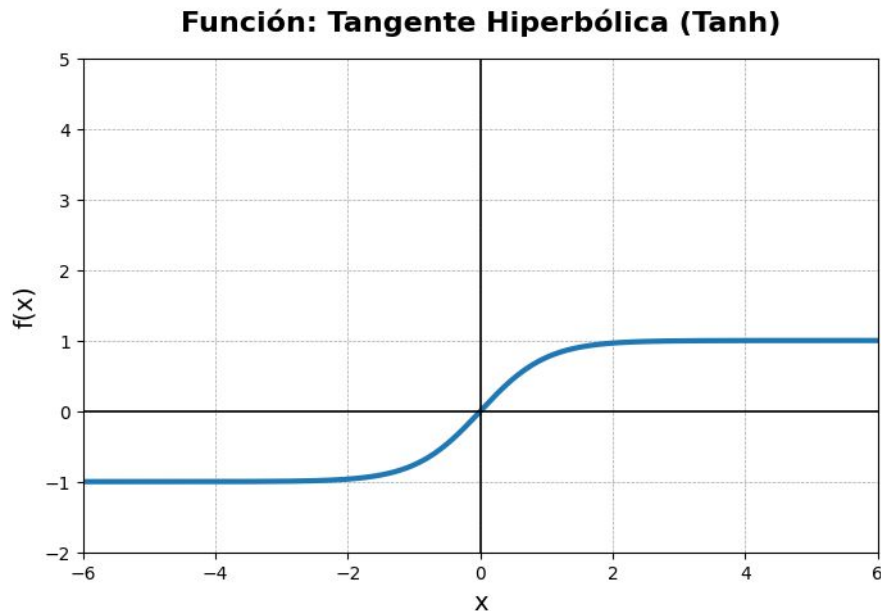
Convergence is faster not only if the inputs are shifted as described above but also if they are scaled so that all have about the same covariance, C_i , where

$$C_i = \frac{1}{P} \sum_{p=1}^P (z_i^p)^2. \quad (13)$$

Extracto de "Efficient BackProp" de LeCun et al.

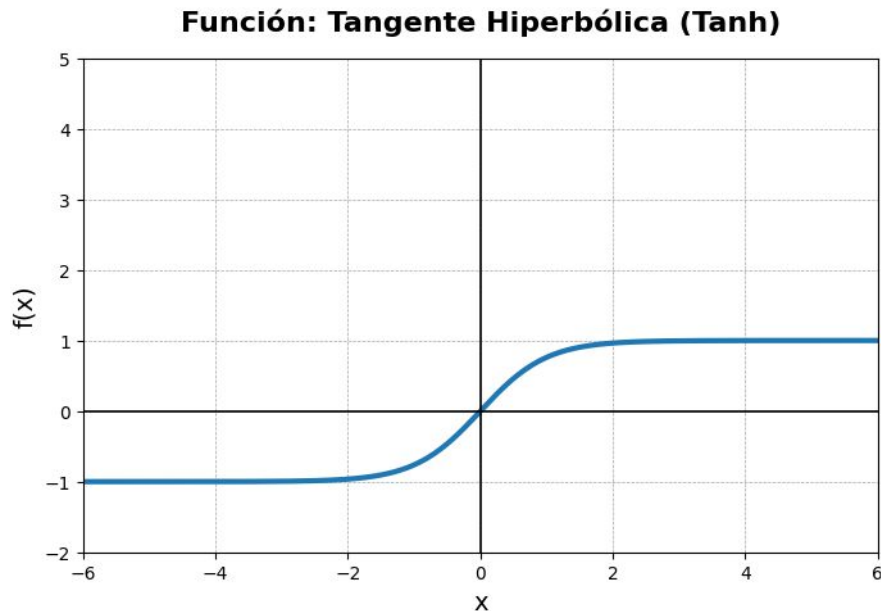
Tangente Hiperbólica (TanH)

- Definición: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Cuándo: Finales de los 80/Principios 90
- Por qué: Para tener la salida centrada en el 0.



Tangente Hiperbólica (TanH)

- Ventajas:
 - Centrada en cero, convergencia de GD más rápida.
 - Sigue estando acotada, evitando problemas.
- Desventajas:
 - En los extremos sigue sufriendo *vanishing gradients*.
 - Más exponenciales para calcular.
- Alternativas: Surgieron para evitar *vanishing gradients*.



Vanishing Gradients

En los extremos de la funciones como la sigmoidea o tanh, las derivadas son casi nulas.

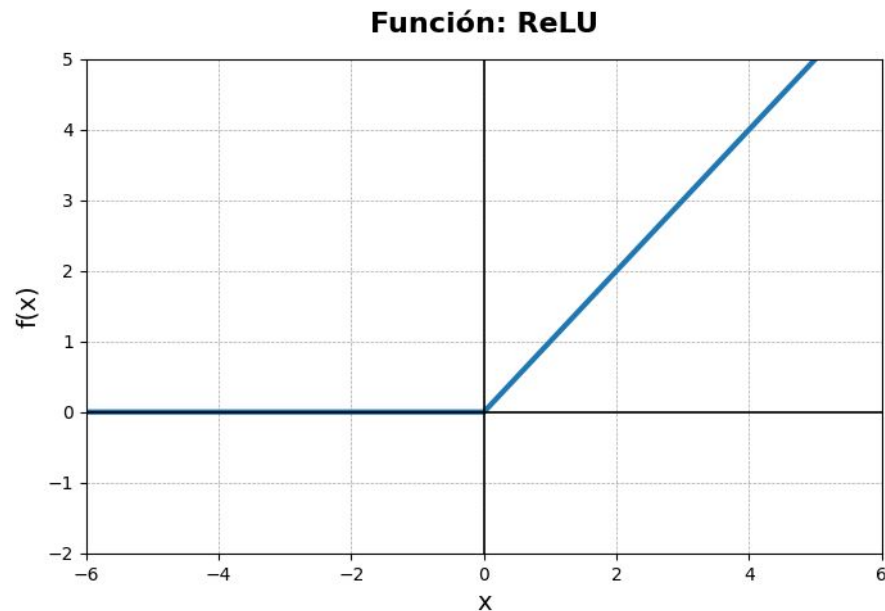
Como vimos en backpropagation, la regla de la cadena de derivadas hace que estas derivadas se vayan multiplicando cada vez que nos movemos hacia las primeras capas.

En redes profundas, los gradientes comienzan a desaparecer y las primeras capas casi no se entrenan.

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[3]}} \frac{\partial a^{[3]}}{\partial z^{[3]}} \frac{\partial z^{[3]}}{\partial a^{[2]}} \frac{\partial a^{[2]}}{\partial z^{[2]}} \frac{\partial z^{[2]}}{\partial W^{[2]}}$$

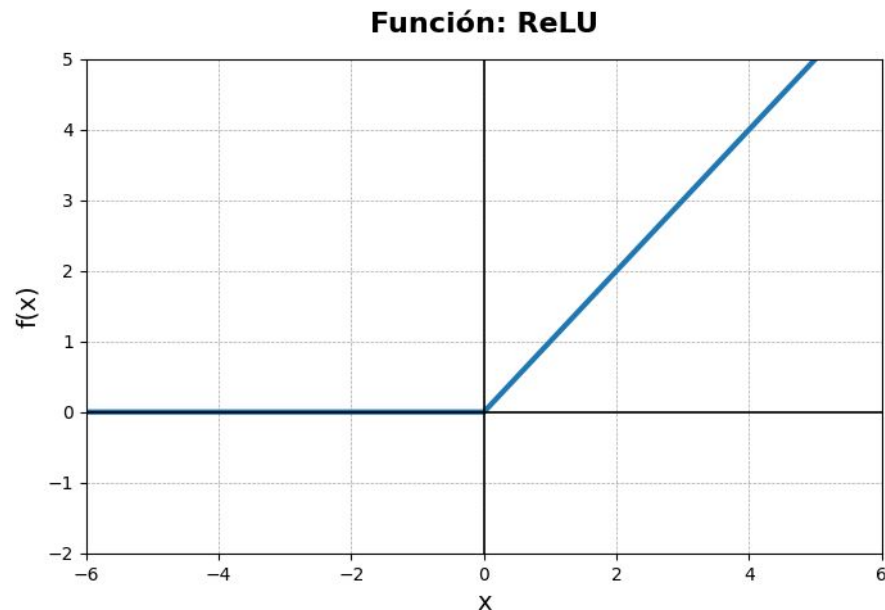
Rectified Linear Unit (ReLU)

- Definición: $f(x) = \max(0, x)$
- Cuándo: Por 2010.
- Por qué: *vanishing gradients* en redes muy profundas.



Rectified Linear Unit (ReLU)

- Ventajas:
 - Derivada siempre igual a 1 en la parte positiva.
 - Apaga ciertas neuronas y eso puede ser útil.
 - Barata computacionalmente.
- Desventajas:
 - La salida no está acotada.
 - Otra vez no está centrada en 0.*
- Alternativas: para resolver el problema de *dying ReLu*.



Dying ReLu

Tener una región en donde la función y su derivada son completamente 0 tiene fuertes consecuencias en el entrenamiento de la red.

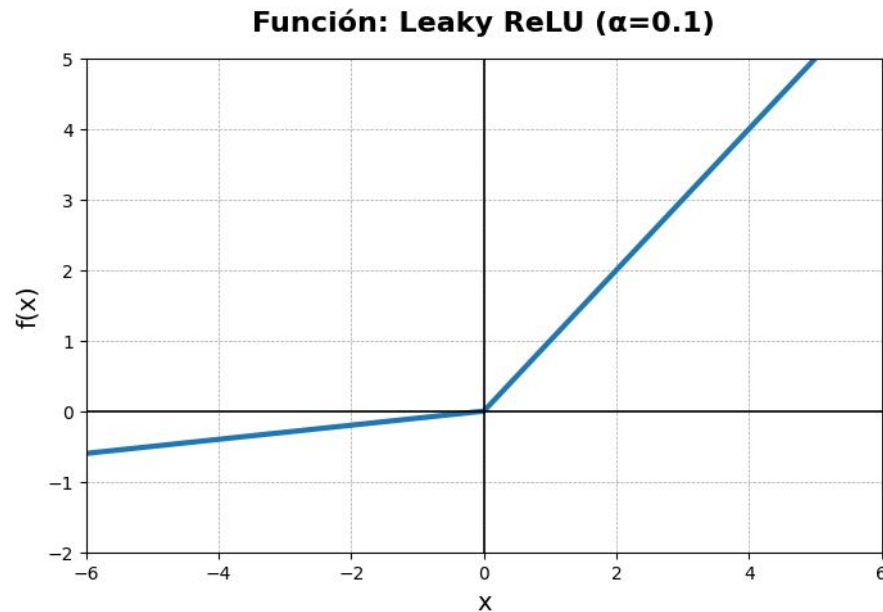
Esta característica produce una gran cantidad de *neuronas muertas*. Son neuronas que nunca se activan y que además nunca se actualizan.

La característica de *Dying ReLU* no es necesariamente un problema porque introduce una fuerte esparcidad en la red que tiene algunas propiedades deseables como el aprendizaje selectivo en las zonas más determinantes.

En algunos casos, con una inicialización inadecuada de los pesos, el problema se puede presentar en un porcentaje tan alto de neuronas que hacen inútil el entrenamiento.

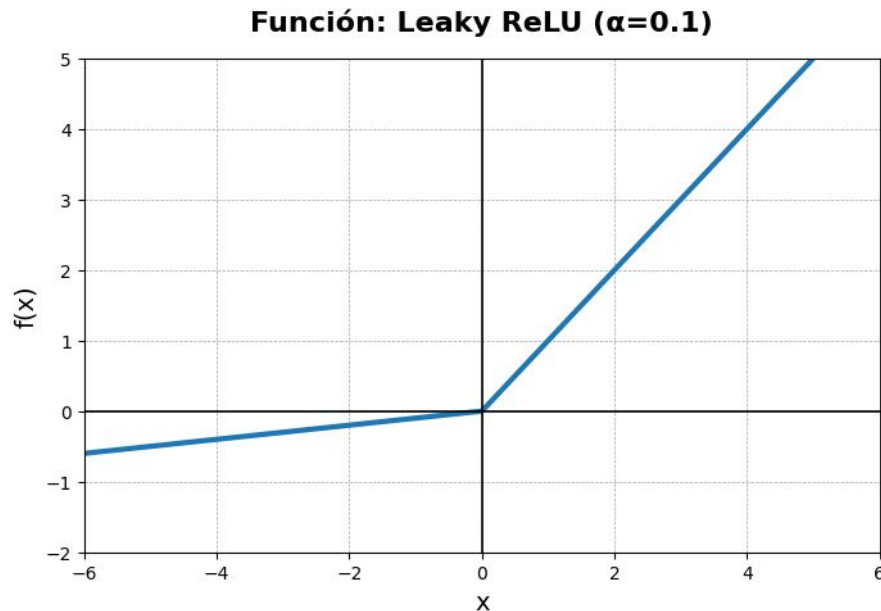
Leaky ReLU

- Definición: $f(x) = x$ si $x > 0$, de lo contrario αx
- Cuándo: 2013
- Por qué: para resolver *dying ReLU*.



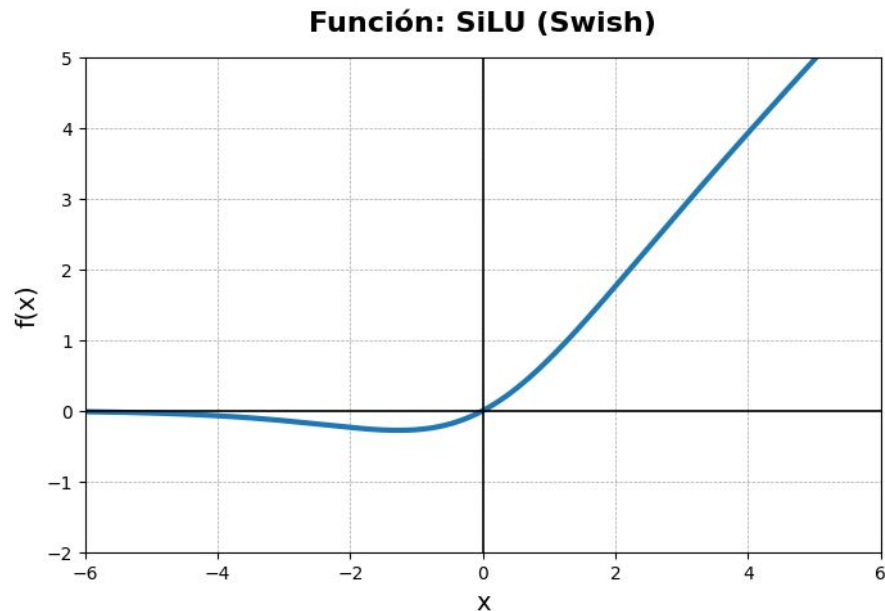
Leaky ReLU

- Ventajas:
 - Resuelve el *dying ReLU*.
 - Sigue siendo computacionalmente *barata*.
- Desventajas:
 - ¿Qué *alfa* se usa?
 - No necesariamente anda mejor que ReLU en la práctica.
- Alternativas: la no suavidad no es muy deseada.



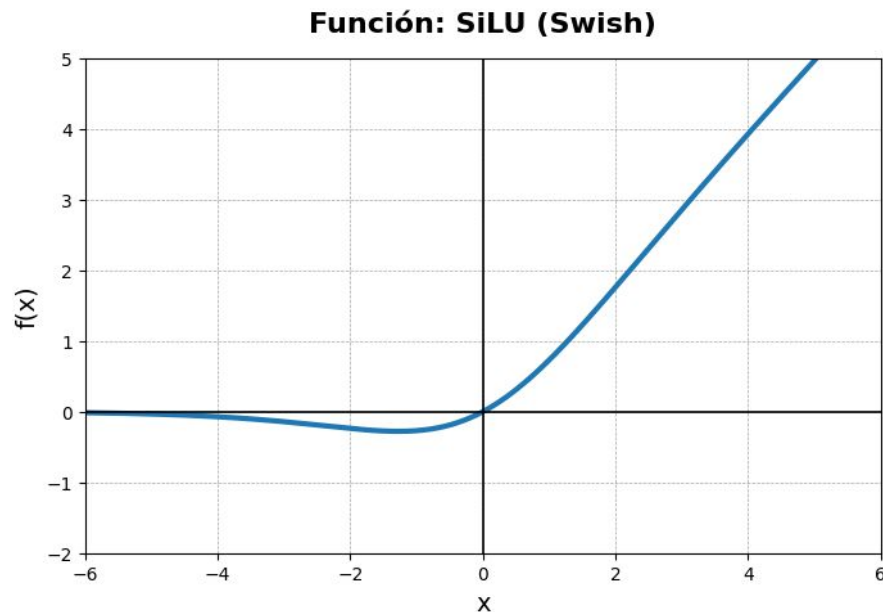
Swish (SiLU)

- Definición: $f(x) = x \cdot \sigma(x)$
- Cuándo: 2016
- Por qué: búsqueda automatizada de funciones de activación.



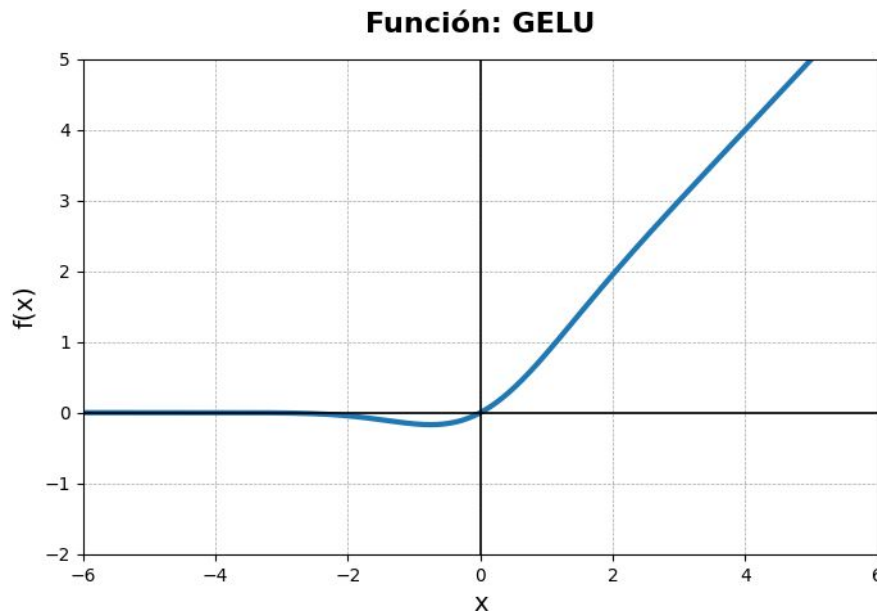
Swish (SiLU)

- Ventajas:
 - Continua, diferenciable, suave.
 - El *vallecito* tiene un efecto de regularización.
 - Para redes profundas suele funcionar mejor que ReLU.
- Desventajas:
 - Es un poco más costosa computacionalmente.
 - Pierde la posibilidad de esparcidad que daba ReLU (a veces deseable).
- Alternativas: más que nada van surgiendo alternativas *de nicho*



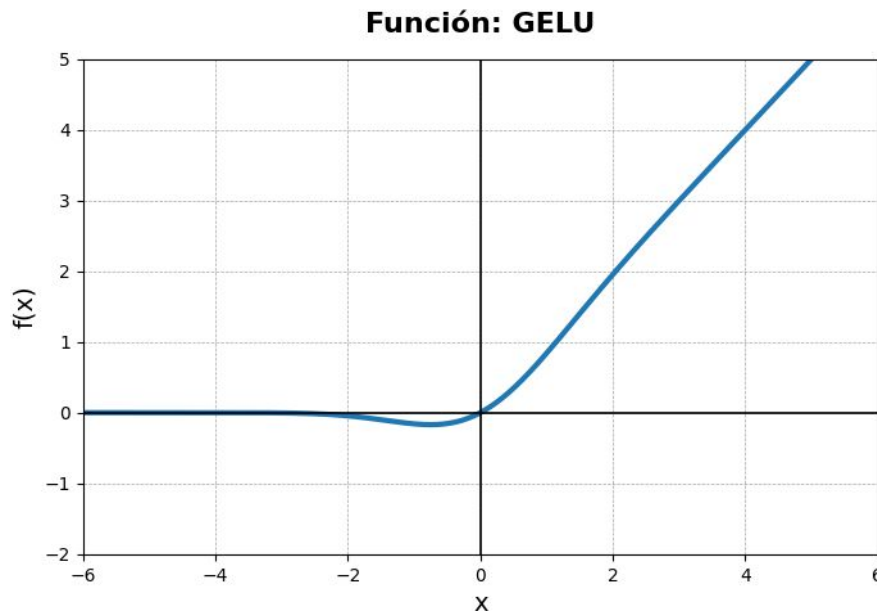
Gaussian Error Linear Unit (GELU)

- Definición: $f(x) = x \cdot \Phi(x)$
- Cuándo: 2016
- Por qué: búsqueda automatizada mezclando conceptos.



Gaussian Error Linear Unit (GELU)

- Ventajas:
 - Suele funcionar muy bien con Transformers.
 - Propiedades teóricas deseables por su definición.
- Desventajas:
 - La formulación exacta es pesada, se usan aproximaciones.
- Alternativas: igual que Swish, más que alternativas, hay competidores de nicho.



Hasta ahora todas las funciones de activación toman un único escalar y lo transforman.

Existe otras funciones (no formalmente de activación) importantes en las NN que transforman las salidas de las capas lineales.

Softmax es una función que toma todo el vector de salidas de una capa y lo transforma en un nuevo vector. Se suele utilizar únicamente en la capa de salida para transformar valores arbitrarios en probabilidades.

Softmax

$$f(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Softmax toma un vector de números y a cada uno de ellos lo exponencia para luego normalizar todos los valores.

Se transforman en probabilidades porque por definición (al estar normalizando) todos los valores de salida posible van a sumar 1.

Su definición está pensada para exacerbar el mayor valor de todo el vector (al estar pensado para problemas de clasificación *single class*).

- Ventajas:
 - La salida es fácilmente interpretable y utilizable como probabilidad de la clasificación.
 - *Winner-take-it-all (suave)*: Si hay un claro ganador, lo destaca, sin anular completamente al resto.
 - Los exponenciales en su definición se *llevan bien* con los logaritmos de las loss functions como NLL y Cross-Entropy Loss.
- Desventajas:
 - Los exponenciales son inestables para calcular, se suelen usar *truquitos*.
 - Al estar normalizando, los valores *compiten* entre sí, lo cual no es adecuado para problemas *multi-class*.

Softmax

```
import numpy as np
import matplotlib.pyplot as plt

def softmax(logits):
    exp_z = np.exp(logits - np.max(logits))
    return exp_z / np.sum(exp_z)

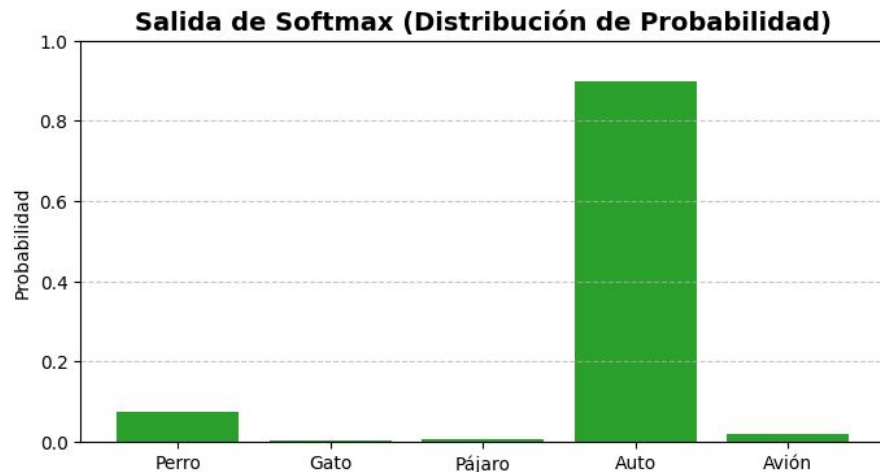
logits = np.array([2.5, -1.0, 0.1, 5.0, 1.2])

probabilidades = softmax(logits)

print(f"Logits crudos: {logits}")
print(f"Probabilidades: {np.round(probabilidades, 3)}")
print(f"Suma total: {np.sum(probabilidades)}")

clases = ['Perro', 'Gato', 'Pájaro', 'Auto', 'Avión']
plt.figure(figsize=(8, 4))
plt.bar(clases, probabilidades, color='#2ca02c')
plt.title('Salida de Softmax (Distribución de Probabilidad)', fontsize=14, fontweight='bold')
plt.ylabel('Probabilidad')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

Logits crudos: [ 2.5 -1.   0.1  5.   1.2]
Probabilidades: [0.074 0.002 0.007 0.897 0.02 ]
Suma total: 1.0
```



Softmax

```
import numpy as np
import matplotlib.pyplot as plt

def softmax(logits):
    exp_z = np.exp(logits - np.max(logits))
    return exp_z / np.sum(exp_z)

logits = np.array([4.8, -1.0, 0.1, 5.0, 3.2])

probabilidades = softmax(logits)

print(f"Logits crudos: {logits}")
print(f"Probabilidades: {np.round(probabilidades, 3)}")
print(f"Suma total: {np.sum(probabilidades)}")

clases = ['Perro', 'Gato', 'Pájaro', 'Auto', 'Avión']
plt.figure(figsize=(8, 4))
plt.bar(clases, probabilidades, color='#2ca02c')
plt.title('Salida de Softmax (Distribución de Probabilidad)', fontsize=14, fontweight='bold')
plt.ylabel('Probabilidad')
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

Logits crudos: [ 4.8 -1.  0.1  5.  3.2]
Probabilidades: [0.411 0.001 0.004 0.502 0.083]
Suma total: 0.9999999999999999
```

